



Stock verifiers

Current PID research checks

Sepehr Mahmoudian

1 October 2026

There is no new formally verified mathematical breakthrough in this update. The work below checks whether our verification route deserves trust. It records bounded upstream byte matches and defects in our own draft code. No Lean, Mathlib, kernel, verifier or scientific acceptance setting was changed. No theorem, control suite or neural experiment was run.

What matched upstream

The active pin remains Lean 4.33.0 at commit `d8b18978322de05a8f3dba51ef03cf5461676c17`, with Mathlib at `db584cd6d46c92f209a44c0f1c829460d327499d`. Current official API responses resolve both commits and the [Lean release](#). These are provider observations, not publisher authentication.

Root compared three complete installed Lean source files with immutable upstream bytes: `Lean/Meta/ExprDefEq.lean`, `Lean/Meta/WHNF.lean` and `Init/Core.lean`. All three SHA-256 values agree. Installed `lean` and `leanchecker` executable hashes agree with the preserved release replay receipt. A separate reviewer also observed matching selected Mathlib source bytes and the declared local Git revision. Same host, model and source dependencies limit review independence.

These observations do not certify the whole installation. Shared libraries, compiled imports, transitive checkouts and the source-to-executed-binary relation remain unqualified. The exact old release-archive locator is absent; no exhaustive search proved that all copies are absent. A failed or partial upstream transfer is not evidence of local modification.

The historical baseline documents seven narrow elaboration compatibility scopes. They are not new changes in this update and are not default-option runs. Their current complete audit remains open. The [official migration note](#) distinguishes the narrow type-transparency option from its broader parent. Future work must not weaken options, axioms or kernels to obtain acceptance; historical evidence keeps its actual settings.

Defects in our draft checks

The draft declaration inspector compares types or definition values before refusing forbidden dependencies. That comparison can reach compiled native reduction first. This is a source-reachable trust-order risk in our inspector, not an observed false proof or a finding that Lean is broken. The remedy belongs in our own pre-comparison checks.

The stopped proof-driver control harness has two further defects. First, it often checks only the failure stage. An unrelated earlier read error at that stage can be counted as the intended negative control. Second, a later persistence failure replaces an earlier classification error and omits the updated





custody ledger from returned records. Its production path correctly refuses missing prerequisites; no production bypass was observed.

The stopped neural extractor and its separate judge have a shared weakness: some original-input joins allow equal integer and floating-point values. Their termination-receipt joins use ordinary Python equality. `Python numeric semantics` explain the distinction: `0 == False` and `1 == 1.0`, although their types differ. Thus `[{'k': 0}]` and `[{'k': False}]` can compare equal. These are manual comparator witnesses, not executed forged trials. Strict comparison of the final candidate output cannot recover a distinction already lost between original inputs.

All original drafts, critiques and failed observations remain preserved. None of these defects has been repaired or qualified in this update. A valid external typed reopening could reject malformed inputs first; that missing premise is not disproved.

Mathematical and application boundaries

The new 26 generic event-law theorem drafts and 16 semantic-control theorem drafts remain uncompiled and kernel-unchecked. They do not close SxPID3 Programs A to E.

The intended research object is finite categorical Makkeh–Gutknecht–Wibral shared exclusions, with a nonnegative normalized joint law, positive evaluated support and nats. It is not continuous Ehrlich PID, KSG, Williams–Beer `I_min` or a context-pooled law. No Gaussian assumption is needed to compute a categorical empirical law; population inference requires a separate sampling model.

The proposed neural pilot has 32 reset rows: 16 training and 16 evaluation, eight per commanded class in each role. Three completed population counts become categories zero through three; the target is binary. The completed window is 19.9 ms, not the requested 20 ms. Independence, actual input custody, native qualification and owner release remain unestablished. Command encoding is not a demonstrated biological or navigation benefit.

Rust implementation and computational cost

No Rust implementation changed or ran. The dated engineering reference describes the default-build categorical API `pid_core::stable::categorical::discrete_sxpid3` and budgeted variants. It does not implement these proof-driver or neural-reopening drafts. For the proposed training input, shape is 16 aligned rows, three categorical sources and one binary target; full capacity is 128 states and occupied support is at most 16. Parsing, reopening, preprocessing, prediction, resampling and their memory costs must be assessed separately. No runtime, peak-memory, real-time or binary64-refinement result is reported.

Next useful milestone

Keep upstream tools unchanged. Establish their intended runtime and import provenance, then check a small, independently specified theorem using ordinary stock verification. Repair our own identity and failure-custody checks in separate reviewed revisions. Real neural or robotics claims require actual data and held-out task-loss comparisons; a polished report, matching hash or passing fixture cannot supply them.





Citation: Sepehr Mahmoudian (2026), *Stock verifiers and current PID research checks*. Record the exact source commit and local changes. Software citation: `CITATION.cff`. This is a private audit draft, not publication acceptance.

